PART B
(PART B: TO BE COMPLETED BY STUDENTS)

| | |
|---|---|
| Roll. No.: C-60 | NAME: Avdhut Madhavi |
| Class: C | Batch:C2 |
| Date of Experiment: | Date of Submission: |
| Grade: | |

**B.1: Read the given question carefully and write SQL statement**

Students are required to **write and execute at least 15 SQL statements** based on **Views and Nested Queries** using their **Lab 1 database.**

The queries should include:

1. Create a view on a single table
2. Display data from the view
3. Create a view using multiple tables
4. Create a view using WHERE clause (MarksView type)
5. Drop a view
6. Single row subquery
7. Subquery using IN
8. Subquery using ANY
9. Subquery using ALL
10. Subquery using EXISTS

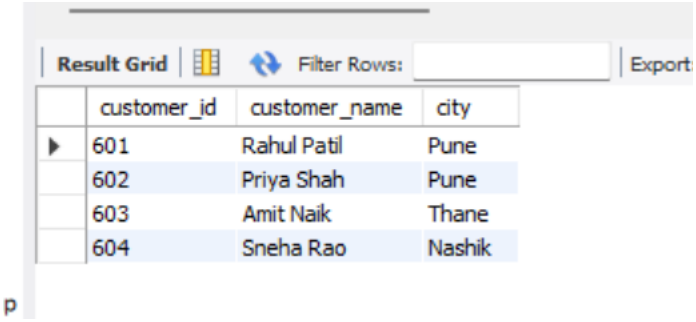
**Aim:** To perform practical implementation of SQL commands for creating views and executing nested queries.

**Database:** Cinema Ticket Booking System (cinema_db)

**1. Create a view on a single table**

```
CREATE VIEW customer_view AS
SELECT customer_id, customer_name, city
FROM customer
WHERE city IS NOT NULL;

SELECT * FROM customer_view;
```



Result Grid | Filter Rows: | Export:

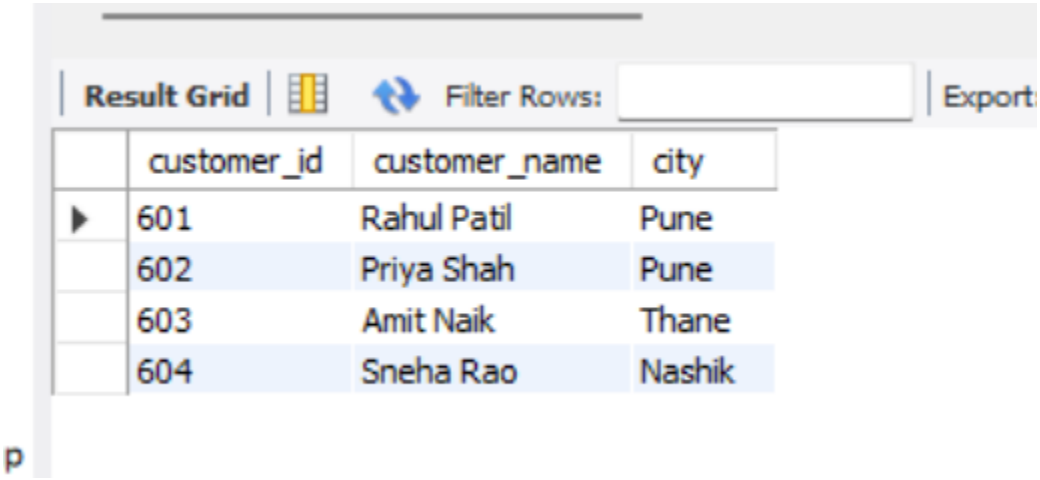
| customer_id | customer_name | city |
|---|---|---|
| 601 | Rahul Patil | Pune |
| 602 | Priya Shah | Pune |
| 603 | Amit Naik | Thane |
| 604 | Sneha Rao | Nashik |

p

**Explanation:** Creates a view displaying customer IDs, names, and cities for customers whose city is not NULL.

**2. Display data from the view**

SELECT * FROM customer_view;



Result Grid | Filter Rows: | Export:

| customer_id | customer_name | city |
|---|---|---|
| 601 | Rahul Patil | Pune |
| 602 | Priya Shah | Pune |
| 603 | Amit Naik | Thane |
| 604 | Sneha Rao | Nashik |

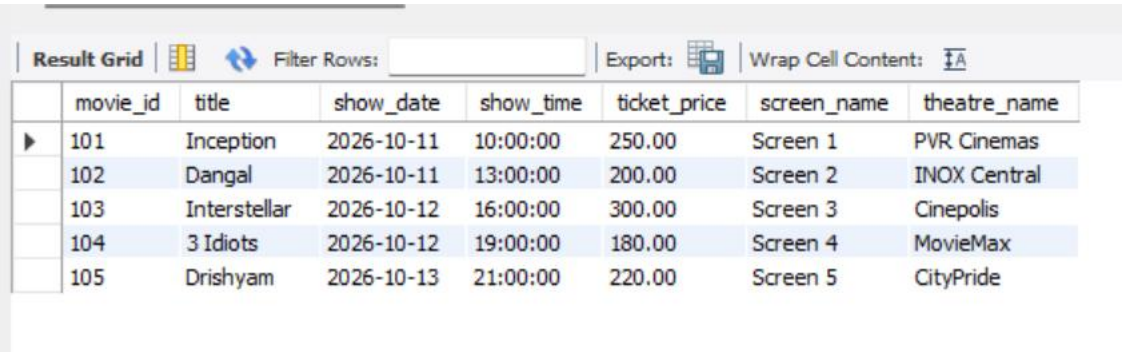
p

**Explanation:** Displays the records returned by customer_view.

**3. Create a view using multiple tables**

```
CREATE VIEW movie_schedule_view AS
SELECT m.movie_id,
    m.title,
    s.show_date,
    s.show_time,
    s.ticket_price,
    sc.screen_name,
    t.theatre_name
FROM movie_details m
JOIN showtime s
```

```
    ON m.movie_id = s.movie_id
JOIN screen sc
    ON s.screen_id = sc.screen_id
JOIN theatre t
    ON sc.theatre_id = t.theatre_id;
SELECT * FROM movie_schedule_view;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

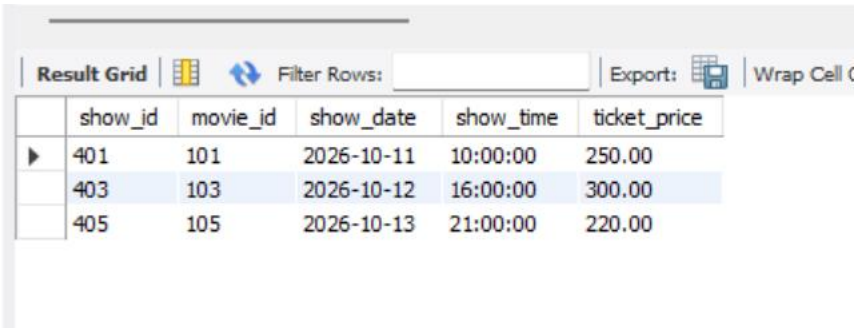
| movie_id | title | show_date | show_time | ticket_price | screen_name | theatre_name |
|---|---|---|---|---|---|---|
| 101 | Inception | 2026-10-11 | 10:00:00 | 250.00 | Screen 1 | PVR Cinemas |
| 102 | Dangal | 2026-10-11 | 13:00:00 | 200.00 | Screen 2 | INOX Central |
| 103 | Interstellar | 2026-10-12 | 16:00:00 | 300.00 | Screen 3 | Cinepolis |
| 104 | 3 Idiots | 2026-10-12 | 19:00:00 | 180.00 | Screen 4 | MovieMax |
| 105 | Drishyam | 2026-10-13 | 21:00:00 | 220.00 | Screen 5 | CityPride |

**Explanation:** Combines movie, showtime, screen, and theatre details into one view.

**4. Create a view using a WHERE clause**

```
CREATE VIEW expensive_show_view AS
SELECT show_id, movie_id, show_date, show_time, ticket_price
FROM showtime
WHERE ticket_price > 200;

SELECT * FROM expensive_show_view;
```



Result Grid | Filter Rows: | Export: | Wrap Cell

| show_id | movie_id | show_date | show_time | ticket_price |
|---|---|---|---|---|
| 401 | 101 | 2026-10-11 | 10:00:00 | 250.00 |
| 403 | 103 | 2026-10-12 | 16:00:00 | 300.00 |
| 405 | 105 | 2026-10-13 | 21:00:00 | 220.00 |

**Explanation:** Creates a view containing only shows with ticket prices above 200.

**5. Drop a view**

DROP VIEW expensive_show_view;

| | | | |
|---|---|---|---|
| 101 | 23:10:53 | CREATE VIEW expensive_show_view AS SELECT show_id, movie_id, show_date, s... | 0 row(s) affected |
| 102 | 23:11:40 | SELECT * FROM expensive_show_view LIMIT 0, 1000 | 3 row(s) returned |
| 103 | 23:12:45 | DROP VIEW expensive_show_view | 0 row(s) affected |

**Explanation:** Removes the view definition. It does not delete records from the original showtime table.

**6. Single-row subquery: Find the show with the highest ticket price**

```
SELECT show_id, movie_id, ticket_price
FROM showtime
WHERE ticket_price = (
   SELECT MAX(ticket_price)
   FROM showtime
);
```

Result Grid | Filter Rows:

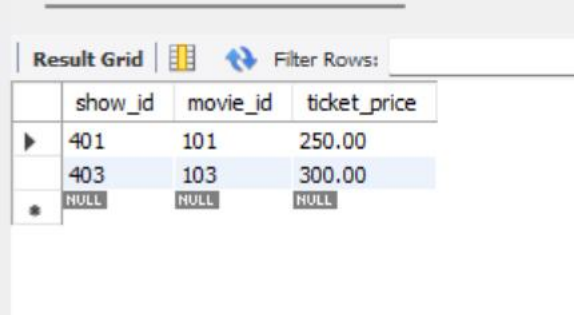
| show_id | movie_id | ticket_price |
|---|---|---|
| 403 | 103 | 300.00 |
| NULL | NULL | NULL |

up

**Explanation:** The inner query finds the maximum ticket price. The outer query displays the show or shows with that price.

**7. Single-row subquery: Find shows priced above the average**

```
SELECT show_id, movie_id, ticket_price
FROM showtime
WHERE ticket_price > (
   SELECT AVG(ticket_price)
   FROM showtime
);
```



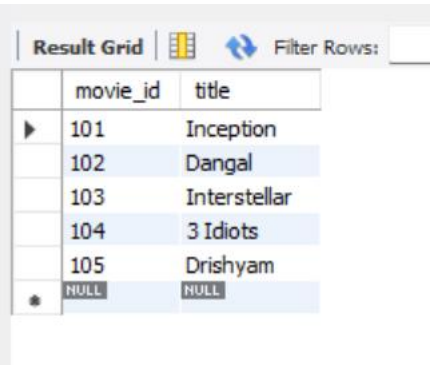
Result Grid | Filter Rows:

| show_id | movie_id | ticket_price |
|---|---|---|
| 401 | 101 | 250.00 |
| 403 | 103 | 300.00 |
| NULL | NULL | NULL |

**Explanation:** Displays shows whose ticket prices are greater than the average ticket price.

**8. Subquery using IN: Find movies that have scheduled shows**

```
SELECT movie_id, title
FROM movie_details
WHERE movie_id IN (
   SELECT movie_id
   FROM showtime
);
```



Result Grid | Filter Rows:

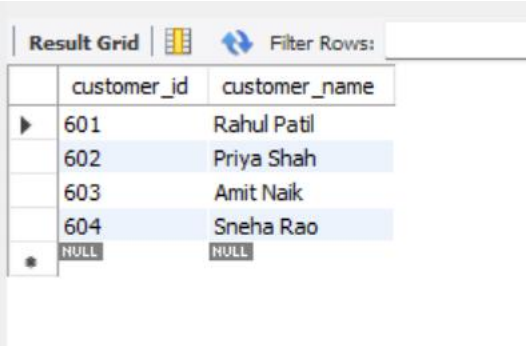
| movie_id | title |
|---|---|
| 101 | Inception |
| 102 | Dangal |
| 103 | Interstellar |
| 104 | 3 Idiots |
| 105 | Drishyam |
| NULL | NULL |

**Explanation:** Displays movies whose IDs appear in the showtime table.

**9. Subquery using IN: Find customers who have bookings**

```
SELECT customer_id, customer_name
FROM customer
WHERE customer_id IN (
   SELECT customer_id
   FROM booking
```

);



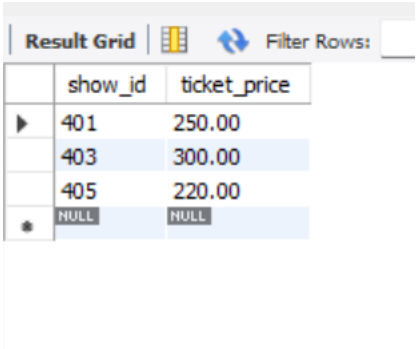
Result Grid | Filter Rows:

| customer_id | customer_name |
|---|---|
| 601 | Rahul Patil |
| 602 | Priya Shah |
| 603 | Amit Naik |
| 604 | Sneha Rao |
| NULL | NULL |

**Explanation:** Displays customers whose IDs appear in the booking table.

**10. Subquery using ANY: Find shows priced above at least one show on October 11**

```
SELECT show_id, ticket_price
FROM showtime
WHERE ticket_price > ANY (
   SELECT ticket_price
   FROM showtime
   WHERE show_date = '2026-10-11'
);
```



Result Grid | Filter Rows:

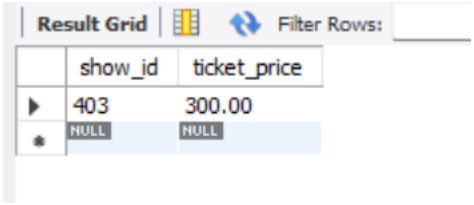
| show_id | ticket_price |
|---|---|
| 401 | 250.00 |
| 403 | 300.00 |
| 405 | 220.00 |
| NULL | NULL |

**Explanation**: Returns shows whose ticket price is greater than at least one ticket price among the shows scheduled on October 11, 2026.

**11. Subquery using ALL: Find shows priced above every show on October 11**

```
SELECT show_id, ticket_price
FROM showtime
WHERE ticket_price > ALL (
```

```
    SELECT ticket_price
    FROM showtime
    WHERE show_date = '2026-10-11'
);
```

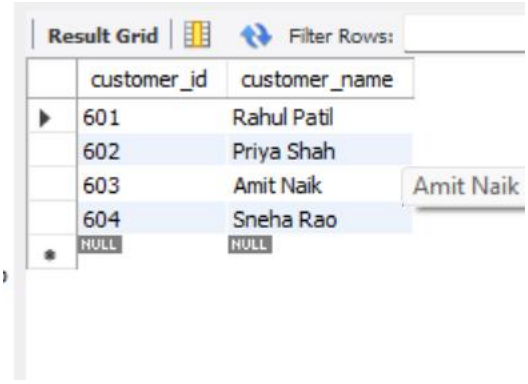


| show_id | ticket_price |
|---|---|
| 403 | 300.00 |
| NULL | NULL |

**Explanation:** Returns shows whose ticket price is greater than every ticket price returned by the inner query.

**12. Subquery using EXISTS: Find customers who have bookings**

```
SELECT c.customer_id, c.customer_name
FROM customer c
WHERE EXISTS (
    SELECT 1
    FROM booking b
    WHERE b.customer_id = c.customer_id
);
```



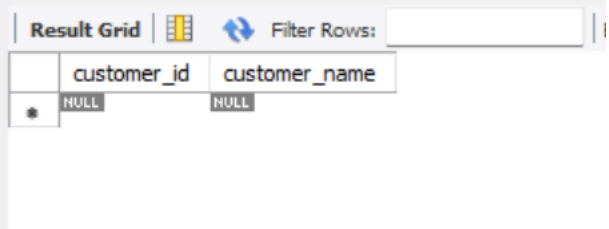
| customer_id | customer_name |
|---|---|
| 601 | Rahul Patil |
| 602 | Priya Shah |
| 603 | Amit Naik |
| 604 | Sneha Rao |
| NULL | NULL |

**Explanation:** Displays customers for whom at least one matching booking record exists.

**13. Subquery using NOT EXISTS: Find customers who have not made bookings**

```
SELECT c.customer_id, c.customer_name
FROM customer c
WHERE NOT EXISTS (
```

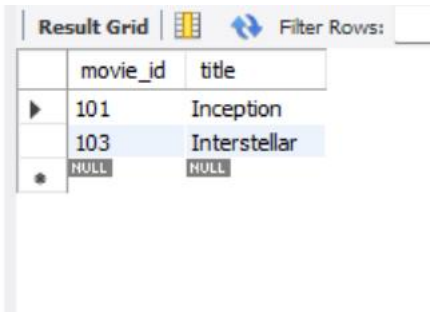
```
    SELECT 1
    FROM booking b
    WHERE b.customer_id = c.customer_id
);
```




**Explanation:** Displays customers who have no matching records in the booking table.

**14. Nested query: Find movies with above-average ticket prices**

```
SELECT movie_id, title
FROM movie_details
WHERE movie_id IN (
    SELECT movie_id
    FROM showtime
    WHERE ticket_price > (
        SELECT AVG(ticket_price)
        FROM showtime
    )
);
```

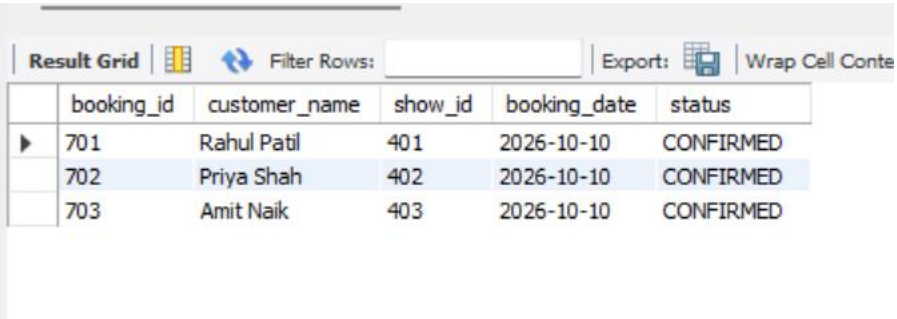



**Explanation:** The innermost query calculates the average ticket price. The next query finds movie IDs with prices above that average, and the outer query displays their titles.

**15. Create a view for confirmed bookings**

```
CREATE VIEW confirmed_booking_view AS
SELECT b.booking_id,
       c.customer_name,
       b.show_id,
       b.booking_date,
       b.status
FROM booking b
JOIN customer c
   ON b.customer_id = c.customer_id
WHERE b.status = 'CONFIRMED';

SELECT * FROM confirmed_booking_view;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Conte

| booking_id | customer_name | show_id | booking_date | status |
|---|---|---|---|---|
| 701 | Rahul Patil | 401 | 2026-10-10 | CONFIRMED |
| 702 | Priya Shah | 402 | 2026-10-10 | CONFIRMED |
| 703 | Amit Naik | 403 | 2026-10-10 | CONFIRMED |

**Explanation:** Creates a view containing confirmed bookings and the corresponding customer names.

**B.2: Observation**

Write the observation based on the execution of the above queries.

1. Views were created using the CREATE VIEW statement to display selected information from the Cinema Ticket Booking System database.
2. Data was retrieved from views using the SELECT statement.
3. A view using multiple tables was created by applying JOIN operations.
4. The DROP VIEW statement was used to remove a view.
5. Single-row subqueries were used to find maximum and average ticket prices.
6. The IN operator was used to identify movies with scheduled shows and customers with bookings.
7. The ANY and ALL operators were used to compare ticket prices with values returned by subqueries.

8. The EXISTS and NOT EXISTS operators were used to identify customers with and without bookings.
9. Nested queries were used to retrieve movies based on average ticket prices.

**Result:** Views and nested queries were implemented and practised using MySQL Workbench.

**B.3: Viva Questions Viva Answers**

1. What is a View?

   A view is a virtual table created using a SQL query. It displays selected data from one or more underlying tables.

   Example:

```
CREATE VIEW customer_view AS
SELECT customer_id, customer_name
FROM customer;
```

2. Does a View store data?

   A normal view does not store its own copy of the query results. It retrieves data from the underlying table or tables when queried.

3. What is a Subquery?

   A subquery is a SQL query written inside another SQL query. It provides a result that the outer query can use.

   Example:

```
SELECT show_id, ticket_price
FROM showtime
WHERE ticket_price > (
   SELECT AVG(ticket_price)
   FROM showtime
);
```

4. What is the use of the IN operator?

   The IN operator checks whether a value matches any value in a list or the result returned by a subquery.

   Example:

```
SELECT movie_id, title
FROM movie_details
WHERE movie_id IN (
   SELECT movie_id
   FROM showtime
);
```

5. What does EXISTS do?

   The EXISTS operator checks whether a subquery returns at least one row. It evaluates to true when a matching row exists.

Example:

```
SELECT c.customer_name
FROM customer c
WHERE EXISTS (
    SELECT 1
    FROM booking b
    WHERE b.customer_id = c.customer_id
);
```